

CS302 Lab 6: King Nala's Grand Feast

Student Name: Manveer Anand

Student ID: 202351080

Course: CS302 - Distributed and Parallel Computing

Lab: 6

Platform: Google Colab (CUDA C++ / NVCC)

This notebook now uses **CUDA C++** for the assignment solution:

- C++ program written to a `.cu` file,
- compilation via `nvcc`,
- execution on GPU,
- assignment outputs printed from the CUDA C++ program.

```
# Colab GPU + NVCC check
!nvidia-smi
!nvcc --version
```

Mon Mar 16 10:06:24 2026

NVIDIA-SMI 580.82.07				Driver Version: 580.82.07		CUDA Version: 13.0	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.	MIG M.
0	Tesla T4	Off	00000000:00:04.0	Off	0	0	
N/A	52C	P8	12W / 70W	0MiB / 15360MiB	0%	Default	N/A

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	
	ID	ID				Usage	
No running processes found							

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Fri_Feb_21_20:23:50_PST_2025
Cuda compilation tools, release 12.8, V12.8.93
Build cuda_12.8.r12.8/compiler.35583870_0
```

Problem Analysis (CUDA C++)

The assignment defines:

$$\begin{aligned} \text{raw}[i] &= (37i + 13) \bmod 101 \\ \text{cooked}[i] &= (\text{raw}[i] \cdot K + 59) \bmod 1009 \\ \text{satisfaction}[i] &= \text{cooked}[i] \bmod 101 \end{aligned}$$

for $N = 2^{22} = 4,194,304$ guests.

A direct brute force over all $K \in [1, 10^6]$ and all guests is extremely expensive, so the CUDA C++ program uses:

1. the required full-guest kernels for verification and unhappy count,
2. a compressed 101-bin search kernel for practical key scanning,
3. host-side reporting of all requested assignment values.

Important math fact used in the report section:

- the total depends on `K mod 1009`, so multiple keys can match the same target total.

```
%writefile king_nala_lab6.cu
#include <cuda_runtime.h>
#include <device_launch_parameters.h>
```

```

#include <chrono>
#include <cstdlib>
#include <cstdlib>
#include <iostream>
#include <vector>

using ll = long long;

constexpr int N = 1 << 22;          // 2^22
constexpr int MOD_RAW = 101;
constexpr int MOD_COOKED = 1009;
constexpr int KEY_MIN = 1;
constexpr int KEY_MAX = 1000000;

__global__ void computeSatisfactionSum(int K, ll* partialSums)
{
    __shared__ ll blockSum[256];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    ll local = 0;
    for (int i = idx; i < N; i += stride)
    {
        int raw = (i * 37 + 13) % MOD_RAW;
        int cooked = (raw * K + 59) % MOD_COOKED;
        int sat = cooked % MOD_RAW;
        local += sat;
    }

    blockSum[tid] = local;
    __syncthreads();

    for (int offset = blockDim.x / 2; offset > 0; offset >>= 1)
    {
        if (tid < offset)
            blockSum[tid] += blockSum[tid + offset];
        __syncthreads();
    }

    if (tid == 0)
        partialSums[blockIdx.x] = blockSum[0];
}

__global__ void countUnhappyGuests(int K, ll* partialCounts)
{
    __shared__ ll blockCount[256];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    ll local = 0;
    for (int i = idx; i < N; i += stride)
    {
        int raw = (i * 37 + 13) % MOD_RAW;
        int cooked = (raw * K + 59) % MOD_COOKED;
        int sat = cooked % MOD_RAW;
        if (sat < 40)
            local++;
    }

    blockCount[tid] = local;
    __syncthreads();

    for (int offset = blockDim.x / 2; offset > 0; offset >>= 1)
    {
        if (tid < offset)
            blockCount[tid] += blockCount[tid + offset];
        __syncthreads();
    }

    if (tid == 0)
        partialCounts[blockIdx.x] = blockCount[0];
}

__global__ void searchKeysCompressed(const ll* rawCounts, ll* totals)
{

```

```

int idx = blockIdx.x * blockDim.x + threadIdx.x;
int K = idx + 1;

if (K <= KEY_MAX)
{
    ll total = 0;
    for (int raw = 0; raw < MOD_RAW; raw++)
    {
        int cooked = (raw * K + 59) % MOD_COOKED;
        int sat = cooked % MOD_RAW;
        total += rawCounts[raw] * sat;
    }
    totals[idx] = total;
}
}

static ll compressedTotal(const std::vector<ll>& rawCounts, int K)
{
    ll t = 0;
    for (int raw = 0; raw < MOD_RAW; raw++)
    {
        int cooked = (raw * K + 59) % MOD_COOKED;
        int sat = cooked % MOD_RAW;
        t += rawCounts[raw] * sat;
    }
    return t;
}

int main()
{
    const int studentId = 202351080;
    const int last4 = studentId % 10000;
    const int Ktrue = 1 + ((last4 * 101) % 1000000);
    const int G = (last4 % 1000) + 1;

    std::vector<ll> rawCounts(MOD_RAW, N / MOD_RAW);
    for (int i = 0; i < (N % MOD_RAW); i++)
        rawCounts[(i * 37 + 13) % MOD_RAW]++;

    ll T = compressedTotal(rawCounts, Ktrue);

    int rawG = (G * 37 + 13) % MOD_RAW;
    int cookedG = (rawG * Ktrue + 59) % MOD_COOKED;
    int satG = cookedG % MOD_RAW;

    ll* dRawCounts = nullptr;
    ll* dTotals = nullptr;

    cudaMalloc(&dRawCounts, MOD_RAW * sizeof(ll));
    cudaMalloc(&dTotals, KEY_MAX * sizeof(ll));
    cudaMemcpy(dRawCounts, rawCounts.data(), MOD_RAW * sizeof(ll), cudaMemcpyHostToDevice);

    const int searchThreads = 256;
    const int searchBlocks = (KEY_MAX + searchThreads - 1) / searchThreads;

    auto t0 = std::chrono::high_resolution_clock::now();
    searchKeysCompressed<<<searchBlocks, searchThreads>>>(dRawCounts, dTotals);
    cudaDeviceSynchronize();
    auto t1 = std::chrono::high_resolution_clock::now();

    std::vector<ll> totals(KEY_MAX);
    cudaMemcpy(totals.data(), dTotals, KEY_MAX * sizeof(ll), cudaMemcpyDeviceToHost);

    int firstFoundK = -1;
    ll lessCount = 0;
    ll greaterCount = 0;
    ll matchCount = 0;

    for (int i = 0; i < KEY_MAX; i++)
    {
        ll val = totals[i];
        int K = i + 1;
        if (val == T)
        {
            if (firstFoundK == -1)
                firstFoundK = K;
            matchCount++;
        }
    }
}

```

```

    }
    else if (val < T)
        lessCount++;
    else
        greaterCount++;
}

// Required kernels with discovered key (use Ktrue for student-specific validation)
const int guestThreads = 256;
const int guestBlocks = (N + guestThreads - 1) / guestThreads;

ll* dPartialSums = nullptr;
ll* dPartialCounts = nullptr;
cudaMalloc(&dPartialSums, guestBlocks * sizeof(ll));
cudaMalloc(&dPartialCounts, guestBlocks * sizeof(ll));

auto k0 = std::chrono::high_resolution_clock::now();
computeSatisfactionSum<<<guestBlocks, guestThreads>>>(Ktrue, dPartialSums);
countUnhappyGuests<<<guestBlocks, guestThreads>>>(Ktrue, dPartialCounts);
cudaDeviceSynchronize();
auto k1 = std::chrono::high_resolution_clock::now();

std::vector<ll> partialSums(guestBlocks), partialCounts(guestBlocks);
cudaMemcpy(partialSums.data(), dPartialSums, guestBlocks * sizeof(ll), cudaMemcpyDeviceToHost);
cudaMemcpy(partialCounts.data(), dPartialCounts, guestBlocks * sizeof(ll), cudaMemcpyDeviceToHost);

ll sumKernel = 0;
ll unhappy = 0;
for (int i = 0; i < guestBlocks; i++)
{
    sumKernel += partialSums[i];
    unhappy += partialCounts[i];
}

double searchSec = std::chrono::duration<double>(t1 - t0).count();
double kernelSec = std::chrono::duration<double>(k1 - k0).count();

std::cout << "Assignment summary\n";
std::cout << "-----\n";
std::cout << "Student ID                : " << studentId << "\n";
std::cout << "Last 4 digits                : " << last4 << "\n";
std::cout << "Ktrue                      : " << Ktrue << "\n";
std::cout << "Target total T              : " << T << "\n";
std::cout << "K found by ascending search : " << firstFoundK << "\n";
std::cout << "Count(total < T)            : " << lessCount << "\n";
std::cout << "Count(total > T)            : " << greaterCount << "\n";
std::cout << "Total matching keys         : " << matchCount << "\n";
std::cout << "Guest number G              : " << G << "\n";
std::cout << "raw[G]                      : " << rawG << "\n";
std::cout << "cooked[G] using Ktrue       : " << cookedG << "\n";
std::cout << "satisfaction[G] using Ktrue : " << satG << "\n";
std::cout << "Guest kernel blocks         : " << guestBlocks << "\n";
std::cout << "Guest kernel threads/block  : " << guestThreads << "\n";
std::cout << "Total launched threads      : " << (1LL * guestBlocks * guestThreads) << "\n";
std::cout << "Search execution time (GPU) : " << searchSec << " seconds\n";
std::cout << "Required kernels exec time  : " << kernelSec << " seconds\n";
std::cout << "Kernel sum == T             : " << (sumKernel == T ? "true" : "false") << "\n";
std::cout << "Unhappy guests (<40)        : " << unhappy << "\n";
std::cout << "Ktrue % MOD mod 1009        : " << (Ktrue % MOD_COOKED) << "\n";
std::cout << "First found K mod 1009      : " << (firstFoundK % MOD_COOKED) << "\n";

cudaFree(dRawCounts);
cudaFree(dTotals);
cudaFree(dPartialSums);
cudaFree(dPartialCounts);

return 0;
}

```

Writing king_nala_lab6.cu

✧ CUDA C++ Program

The cell below writes a full CUDA C++ implementation that includes the required kernels:

- `computeSatisfactionSum`
- `countUnhappyGuests`

and also a practical search kernel:

- `searchKeysCompressed`

Then we compile with `nvcc` and run the executable to produce all assignment outputs.

```
# Compile the CUDA C++ program
!nvcc -O2 king_nala_lab6.cu -o king_nala_lab6
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release
king_nala_lab6.cu(15): warning #177-D: variable "KEY_MIN" was declared but never referenced
    constexpr int KEY_MIN = 1;
                   ^
```

Remark: The warnings can be suppressed with `"-diag-suppress <warning-number>"`

```
# Run the CUDA C++ executable
!./king_nala_lab6
```

```
Assignment summary
-----
Student ID           : 202351080
Last 4 digits        : 1080
Ktrue                : 109081
Target total T       : 209590540
K found by ascending search : 109
Count(total < T)      : 480672
Count(total > T)      : 518337
Total matching keys   : 991
Guest number G       : 81
raw[G]               : 81
cooked[G] using Ktrue : 816
satisfaction[G] using Ktrue : 8
Guest kernel blocks   : 16384
Guest kernel threads/block : 256
Total launched threads : 4194304
Search execution time (GPU) : 0.119454 seconds
Required kernels exec time : 0.000522801 seconds
Kernel sum == T       : true
Unhappy guests (<40)   : 1661109
Ktrue mod 1009        : 109
First found K mod 1009 : 109
```